

AGENT EDUCATION

TÀI LIỆU PHÂN TÍCH & GIẢI THÍCH LUỒNG MÃ NGUỒN

Hệ thống: Trợ lý Học tập AI tích hợp RAG & Đánh giá Tự động

Mô hình ngôn ngữ: Qwen2.5:3b (Chạy cục bộ qua Ollama)

Mô hình nhúng Vector: Nomic Embed Text

Cơ sở dữ liệu Vector: ChromaDB

Tự động khởi tạo bởi: Antigravity AI Assistant

Ngày xuất bản: 2026-07-03

1. Chi tiết Nhiệm vụ Từng File Code

▪ File: `backend/file_processor.py`

Chịu trách nhiệm trích xuất văn bản thô từ tệp tài liệu PDF do học viên tải lên. Thư viện sử dụng chính là 'pdfplumber', hỗ trợ đọc chính xác các lớp văn bản kỹ thuật số.

▪ File: `backend/Model_ai.py`

Quản lý kết nối và giao tiếp với Ollama. Tự động kiểm tra và tải về (pull) các mô hình cần thiết ('qwen2.5:3b' để trả lời và 'nomic-embed-text' để tạo vector). Thiết lập nhiệt độ (temperature=0.2) để AI trả lời bám sát tài liệu gốc và tránh tự sáng tạo.

▪ File: `backend/Rag.py`

Trái tim xử lý tri thức của hệ thống. File này chứa 2 chức năng cực kỳ quan trọng:

- Semantic Chunking: Cắt văn bản theo ngữ nghĩa tự nhiên (đoạn văn, dấu câu) thay vì cắt ký tự cứng để giữ nguyên vẹn thông tin.
- ChromaDB: Kết nối tới CSDL Vector, tạo chỉ mục nhúng (embeddings) và truy vấn K đoạn văn bản tương quan nhất với câu hỏi của học viên.

▪ File: `backend/baomat.py`

Xử lý bảo mật thông tin người dùng bao gồm chức năng Đăng ký và Đăng nhập. Sử dụng thuật toán SHA-256 để mã hóa mật khẩu học viên trước khi lưu xuống file cơ sở dữ liệu dạng text cục bộ.

▪ File: `app.py`

Giao diện người dùng chính được xây dựng bằng thư viện Tkinter. Quản lý việc Đăng nhập/Đăng ký tài liệu học tập, tải lên PDF, bấm Phân tích tài liệu (kích hoạt RAG), hiển thị khung chat thời gian thực và quản lý luồng tương tác giữa học viên với trợ lý ảo.

▪ File: `chay_test_gui.py`

Trình chạy đánh giá kiểm thử giao diện đồ họa (GUI Test Runner). Chạy tuần tự 12 test cases (2 Mock offline và 10 kịch bản RAG thực tế dựa trên sách Data Science). Tính toán tỷ lệ tương đồng ngữ nghĩa Cosine và hiển thị biểu đồ chỉ số kết quả kiểm thử.

▪ File: `chay_test_cmd.py`

Trình chạy kiểm thử dòng lệnh (CMD Test Runner) phục vụ tự động hóa lập trình. Chạy giống phiên bản GUI nhưng in ra terminal với màu sắc trực quan bằng mã màu ANSI, xuất báo cáo cuối cùng và trả về Exit Code (0 nếu pass, 1 nếu fail) cho luồng tích hợp CI/CD.

2. Tổng kết Luồng Hoạt động Hệ thống (System Flows)

Luồng 1: Tải lên và Phân tích tài liệu PDF (Upload & Ingestion Flow)

Bước 1: Học viên đăng nhập vào hệ thống và bấm nút 'Upload File PDF' trên giao diện chính (app.py).

Bước 2: Hệ thống gọi backend/file_processor.py mở file và chuyển toàn bộ văn bản PDF thành chuỗi text thô.

Bước 3: Chuỗi văn bản được đưa vào backend/Rag.py để thực hiện Semantic Chunking (chia nhỏ thành các đoạn có độ dài tối đa 600 ký tự, ngắt câu tự nhiên).

Bước 4: Với mỗi đoạn text, hệ thống gọi backend/Model_ai.py qua Ollama để nhúng (embed) thành vector số thực bằng model 'nomic-embed-text'.

Bước 5: Lưu trữ đồng thời văn bản thô, mã ID và Vector nhúng tương ứng vào CSDL Vector (ChromaDB) để phục vụ tra cứu sau này.

Luồng 2: Học viên gửi câu hỏi & AI trả lời (RAG Query Flow)

Bước 1: Học viên nhập câu hỏi vào ô chat trên ứng dụng app.py và nhấn Enter hoặc nút Gửi.

Bước 2: Câu hỏi được gửi đến lớp RagChroma trong backend/Rag.py.

Bước 3: Lớp này chuyển câu hỏi thành vector nhúng bằng model 'nomic-embed-text'.

Bước 4: ChromaDB thực hiện so sánh khoảng cách Vector và truy xuất ra các đoạn văn bản có độ tương đồng cao nhất (ngữ cảnh tốt nhất).

Bước 5: Lớp RagChroma ghép các đoạn văn bản trên thành một 'Ngữ cảnh tài liệu tham khảo' và chuyển cho Model_AI để hỏi Ollama (sử dụng gwen2.5:3b).

Bước 6: Mô hình AI tổng hợp câu trả lời dựa trên ngữ cảnh được cung cấp và gửi lại cho giao diện chính hiển thị lên màn hình chat.

Luồng 3: Quy trình Đánh giá Kiểm thử (Test Suite Evaluation Flow)

Bước 1: Chạy file chay_test_gui.py (giao diện) hoặc chay_test_cmd.py (dòng lệnh).

Bước 2: Chạy 2 test case đầu tiên (Mock Exception & Mock Context) để xác minh độ ổn định khi sập dịch vụ và cấu trúc của prompt mà không cần kết nối Ollama thực tế.

Bước 3: Chạy 10 test case RAG thực tế. Từng câu hỏi tiếng Việt sẽ được gửi qua RAG lấy ngữ cảnh sách Data Science để sinh câu trả lời thực tế từ AI.

Bước 4: Sử dụng nomic-embed-text tạo vector cho cả câu trả lời thực tế và câu đáp án chuẩn ban đầu.

Bước 5: Tính điểm Cosine Similarity. Nếu độ khớp $\geq 75\%$, đánh giá là ĐẠT (PASSED), ngược lại là THẤT BẠI (FAILED).

Bước 6: Khi kết thúc, tính toán và xuất các chỉ số tham khảo quan trọng: Tỷ lệ đạt, Thời gian thực thi, Độ trùng khớp trung bình, cao nhất và thấp nhất.